

Topic: Topological Sorting Algorithm

What is Topological Sort?

The goal is to create a **linear ordering** of all the nodes in a graph. The rule is: if there is a directed edge from node **U** to node **V** (meaning **U** must come before **V**), then in the final sorted list, **U** will always appear before **V**.

This algorithm only works on a **DAG (Directed Acyclic Graph)**.

- **Directed:** The edges must have a one-way direction ($U \rightarrow V$).
- **Acyclic:** The graph cannot have any cycles (e.g., $A \rightarrow B \rightarrow C \rightarrow A$), as this would create an impossible dependency loop.

Algorithm Steps

This algorithm uses three main components:

1. A **Stack**: This will store our final sorted list (in reverse order).
2. A **Visited Array**: This keeps track of which nodes we have already explored.
3. A **DFS (Depth First Search) Function**: This is the recursive function that explores the graph.

Part 1: The Main Function

This is the starting point that loops through all nodes to ensure every part of the graph is visited.

1. **Initialize:** Create an empty **stack** and a **visited array** (with all entries set to `false`).
2. **Loop:** Iterate through every single node in the graph (e.g., from node 0 to the final node).
3. **Check:** For each node, check if it is in the **visited array**.
 - If the node has **not** been visited, call the **DFS function** (see Part 2) on that node.
 - If the node *has* been visited, do nothing and move to the next node.
4. **Finalize:** After the loop is complete (meaning all nodes have been processed), the **stack** will contain the topologically sorted nodes. To get the final list, simply pop every element from the stack one by one. The order they are popped in is the correct topological sort.

Part 2: The DFS Function (The Recursive Logic)

This is the core of the algorithm. This function is called for a specific `node`.

1. **Mark as Visited:** As soon as you enter the function for a `node`, mark it as `true` in the **visited array**. This prevents you from getting stuck in loops and re-processing the same node.

2. **Explore Neighbors:** Look at all the adjacent nodes (neighbors) that the current `node` has a directed edge to.
3. **Recurse:** For each of those neighbors:
 - Check if the neighbor has been visited.
 - If the neighbor has **not** been visited, make a recursive DFS call on that neighbor. (This new call will start at Step 1 for the neighbor).
 - If the neighbor *has* been visited, do nothing and move to the next neighbor.
4. **Push to Stack:** This is the most important step. **After** you have finished exploring *all* of the current `node`'s neighbors (meaning all recursive calls from Step 3 have completed and returned), push the **current node** onto the **stack**.

Why This Works (The Intuition)

The magic is in **Step 4** of the DFS function. A node is only pushed onto the stack *after* all the nodes it depends on (all the nodes it points to) have been fully explored and pushed onto the stack.

Example: If you have $A \rightarrow B \rightarrow C$:

1. DFS (A) is called. It sees neighbor B.
2. DFS (B) is called. It sees neighbor C.
3. DFS (C) is called. C has no neighbors, so it finishes and **c is pushed onto the stack**.
4. The DFS (B) call resumes. It has no more neighbors, so it finishes and **b is pushed onto the stack**.
5. The DFS (A) call resumes. It has no more neighbors, so it finishes and **a is pushed onto the stack**.

The stack now (from top to bottom) is: **[A, B, C]**.

When you pop from the stack to get the final list, you get: **A**, then **B**, then **C**. This is the correct topological order.

